

UseLoopModel on gfx1250

A schedule model and a dataflow lowering for TensileLite GEMM main loops

Design and implementation review

Every operation in the main loop needs six answers — and the assembly emitter gave all six by position in a Python function



Per op-class, per hop, the decoder must decide:

- at **which loop level** the operation lives
- **how far ahead** of its consumer it runs (retime *off*, peel depth *M*)
- **which rotating buffer** this access touches
- where the **read pointer flips** and the **descriptor advances**
- what **completion counter** it waits on
- where the **fence** goes

Times a parameter space that ships:

- six 3-letter loop orders, plus 6-letter interleaved words (96 accepted spellings, 90 distinct traversals; only the 6 doubled words collapse, so 84 are non-contiguous)
- prefetch depths, tile splits per operand *and* per axis, wave count, bf16 / f8 / MX-scaled f8
- **5,744** bf16, **152** f8, **404–444** mx8 kernels in the shipping matrices

What by-hand looks like when it fails: the region walk had *two owners*. A split operand got four `tensor_loads` where two were needed, both halves shared one memory token, and the increment subtracted an advance it had not caused — the two displacements cancelled only by luck.

The answer is three layers, and each is defined as much by what it is forbidden to know

Layer 1 — LoopModel (LoopIR)

what happens when: presence, retime *off*, peel *M*, ring depths *S* and *W*, read-ahead, the obligation ledger. A *rolled tree*: one node per op-class, buffer indices are formulas.

May not know: addresses, physical registers, wait counts — or any TensileLite Solution key.

rolled tree, symbolic slot expressions

Layer 2 — Lowering (GIR, SSA + CFG)

the physical consequences: which generation an access touches, where the LDS read pointer flips, where the TDM descriptor advances, where fences go. Four textbook fix-points.

May not know: how to re-derive θ . Analyses never mutate the IR, and gir/ imports nothing from the compiled instruction extension.

EmitAction(kind, at) — 8 kinds, resolved integers, no node reference

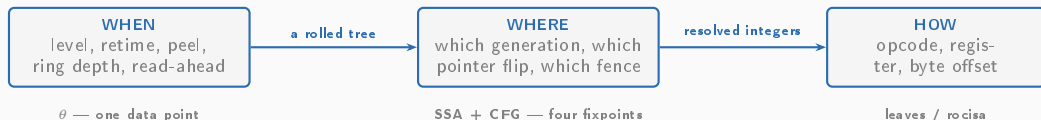
Layer 3 — leaves / rocisa

how: instruction selection and encoding. A buffer swap becomes an `s_xor` on a descriptor, or a `v_xor` on an address register.

May not decide: *what*, or *where*.

Retracted claim: the rule used to read “a grep for MIWaveTile in the core returns nothing”. It hits six core modules. What *does* hold: no core module **reads** a Solution key.

The through-line, and the map: the schedule model decides WHEN, the dataflow IR resolves WHERE



The split is not tidiness. Each layer prevents a failure the other cannot see: a **schedule** question asked of a dataflow pass has no point to reach and no value to propagate; a **placement** question asked of the schedule model is answered by position in a function, which is where every defect in this deck came from.

chapters	layer	the question each answers
2–4 schedule model; placement; hazards	WHEN	what exists, how far ahead it runs, and what it owes
5–7 why a second IR; analyses; passes	WHERE	which buffer, which flip, which fence — and how each is proved
8 address geometry	HOW	the one thing the model is forbidden to know
9–10 what shipped; what is next	—	the measured trajectory, and the named gaps

Decides WHEN — the schedule model

- 2 The schedule model: axes, presence, rings
- 3 Placement: pipelining, read-ahead, peel
- 4 Hazards, obligations, and proof

Resolves WHERE — the dataflow IR

- 5 Why a second IR at all
- 6 The analyses (the technical core)
- 7 Passes, verification, emission

Decides HOW — the leaves

- 8 Address geometry and tile splitting

Evidence

- 9 What shipped, what went wrong, how it is validated
- 10 Gaps, and the bet this design makes



Each layer is forbidden to know the next one's answer.

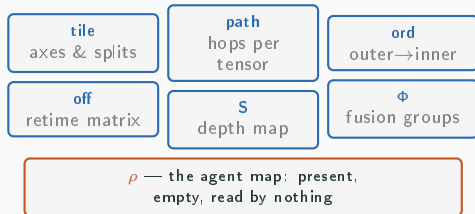
What shipped: the read trajectory, across the whole feature matrix — and a named list of what is still absent

Axis	Status	The measured number
bf16	shipped	5,744-kernel validation matrix
8-bit float	shipped	152 kernels; 152/152 byte-identical emit-diff
MX-scaled fp8	shipped	404–444 kernels; 404/404 byte-identical emit-diff, and 786 pass / 6 fail on the 792-cell hardware matrix
fp6 / fp4	rejected	each has its own local-read packing; the read leaf's fragment table is checked against neither
Loop orders	shipped	six 3-letter words; 6-letter interleaved words require a split
Prefetch / look-ahead	shipped	PGR sets the peel depth M ; hard gate $S \geq \text{PGR}$; PLR read-ahead is per register group
Tile split (A/B)	shipped	free-axis half; shared-K half only with no tail loop; mixed axes since 2026-08-18
Multi-wave	shipped	Φ fuse: several movements, one cooperative instruction, <i>one</i> completion

Not covered, and each one is real work: the epilogue and the whole write side · the tail loop and the edge forks · wave specialization, whose two safety rules are today *structurally inert* · a persistent outer loop · register partitions · a second global-prefetch stage.

The methodological lesson the numbers paid for: the use-before-def gate was written, correct, and called only from the unit tests. A later sweep flagged **216 of 216** single-substep configurations. *An oracle not on the build path is a test, not a gate.*

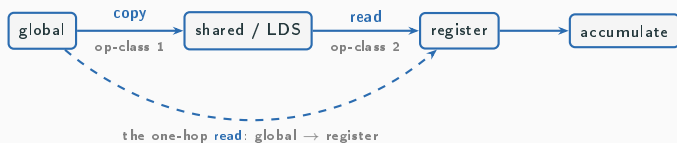
A schedule is seven fields — and deliberately none of them is an address



- **S** is *derived*, not authored.
- **off** is keyed by *op-class* $\times$ *level* — not by operand.
- **Φ** merges op-classes into one cooperative instruction; members pair *by region index*, and the two operands may be cut on different axes.

Address-opacity is an invariant. Addresses, swizzles and per-lane fragment layouts provably change no scheduling decision, so the core treats an address as opaque. `ir.py`'s docstring carries the recipe for re-adding them as an *additive* layer.

Everything is keyed on the op-class — so presence is a per-hop fact, not a per-operand one



An **op-class** = one operand, one hop.

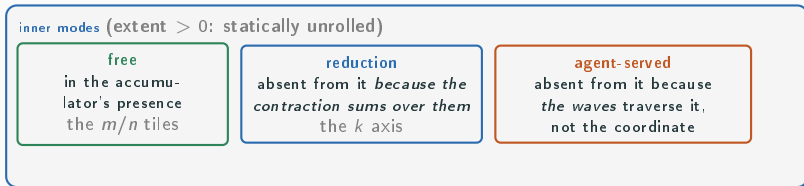
- **role** $\in \{\text{copy, read, store}\}$, from one table.
- **trajectory** is separate: in a store round trip the middle leg's *role* is read while its *sense* is reverse.
- off, the ledger's endpoints and the emitted tree all name op-classes.

Presence = the inner axes an op-class varies over.

hop	presence
bulk cooperative copy	$\emptyset$ (sits at the chunk)
same copy, tile cut into n regions	{region axis}
per-lane read	the operand's data axes

Same operand, different answers. Asking the operand instead of the hop puts a bulk copy at the wrong level — and a region split is what brings the axis back.

Reduction axes are derived by subtraction — which is exactly why a third class was needed



Defined at one point, not at each consumer:

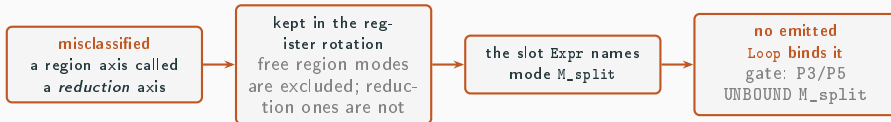
```
reduction = inner
    \ presence(each output op-class)
    \ agent_served
free(op) = presence(op) \ reduction
```

An axis is agent-served when, for some operand:

- it has region modes, *and*
- the region split is > 1 — having an axis is not having two of them, *and*
- some **non-bulk** hop is agent-relative.

The bulk copy still walks every region by index; only the *read* is agent-relative. That asymmetry is the whole class.

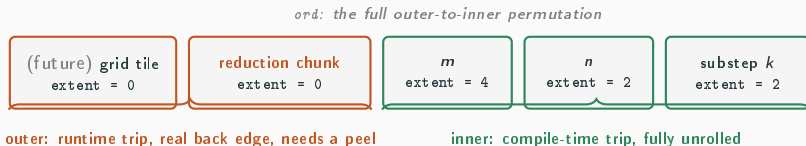
What subtraction cost: a register ring rotating on a variable no loop binds



- Each wave sees exactly *one* value of that axis, so nothing in the read/compute nest rotates over it: the ring had a period nothing advanced.
- **Neither the ring, the gate, nor the slot expression was wrong.** All three did what they were told. The fault was a classification with no name for what the axis actually was.

The guard was written to fail on purpose — and it did. A test asserted, over three configurations, that $\text{inner} \setminus (\text{output presence} \cup \text{reduction}) == \text{agent_served} \cap \text{inner}$ — “any *other* mode leaving the wmma's presence is a new fact, not this one”. It then pins the equivalence that made the change emit-neutral. Evidence: byte-identical output, both codegen suites.

Extent alone tells outer from inner — one field, so a new loop level is not a new decoder branch

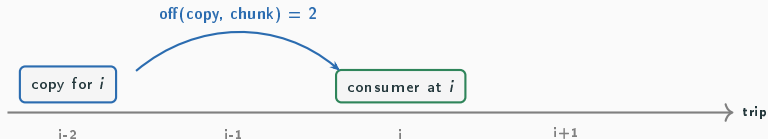


- Shared ring rotates *across* the outer levels; register ring rotates *inside* the inner ones.
- `ord` is a permutation $\Rightarrow$ any subset traverses as a **monotone mixed-radix counter**; a mode's stride is the product of the extents inner to it.

The **reduction chunk** is the *innermost* outer level, and five things key on it: shared ring depth, the read's shared-source rotation, the steady Loop, the Bind the peel pins, and the peel depth at that level.

Before it was centralised, three call sites used `outer_modes()[0]` and two used `[-1]` — identical with one outer level, silently divergent the moment there are two.

Retime is non-negative by type: deferral is a *positive* offset on a reverse hop



```
def off_at(self, opname, role, level_name):
    name = opname if isinstance(opname, str) \
        else opname.name
    return int(self.off_map.get(
        (name, role, level_name), 0))
```

- **Absent means 0** — that op-class is simply not pipelined at that level, so the matrix stays sparse and no point enumerates the full cross product.
- Prefer `off_of(op, hop, level)`: it takes the role *off the hop*. Restating the role at a call site is how a copy's offset gets read under the read's key.

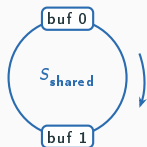
Two entries exist on the shipping parameter path:

<code>off(copy, chunk)</code>	global prefetch depth
<code>off(read, substep)</code>	local prefetch depth

Nested pipelining is just more entries. A copy two chunks *and* one persistent tile ahead is two keys — no decoder change. That is the pay-off for making outer levels real ord modes.

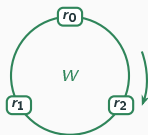
Nothing here enforces the sign. A negative entry is rejected downstream, as ill-typed, with that explanation.

Two rings, one rule: $\text{slot} = \text{coordinate} \bmod S$ — what differs is which coordinate turns



shared / LDS ring

turns once per *trip*: $\text{chunk} \bmod S$



register ring (per group)

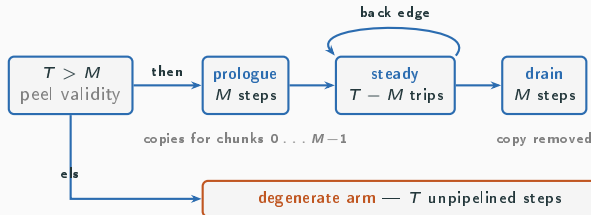
turns inside the *unrolled body*

	shared ring	register ring
coordinate	reduction chunk index	mixed-radix index over the group's <i>rate modes</i>
modulus	S_{shared} : a target fact (the LDS allocation)	W : searched in a derived band $[L, R]$
generation	one chunk of an operand in one buffer	one filling of one register name

- $W < L$ is a race, $W > R$ is waste. $W = 1$ is in-place: safety comes from *ordering*, not capacity.
- Both slots stay symbolic Exprs, so the tree stays rolled.

M (peel depth) is not S_{shared} (buffers). They coincide at prefetch 2 + double-buffered LDS, and diverge at both off-diagonals — *both of which ship*. Deriving $S_{\text{shared}} = \delta$ once emitted *no buffer swap at all*.

Retiming is one integer per op-class; it costs two peels and a fallback arm



- $\text{off}(c, \text{chunk}) = \delta$ means *chunk i's copy is issued in iteration i - δ* . That is the whole of software pipelining, as one integer. $M = \max_p \delta_p$.
- The peel is **staggered, not uniform**: copy_p is present at prologue step t iff $\delta_p \geq M - t$, and at drain step t iff $\delta_p \leq M - 1 - t$ — deepest in first, deepest out first.
- The guard is **strictly** $T > M$. The steady region lowers to a *post-tested* block, which cannot express zero trips; under $\geq$, $T = M$ would run one trip computing chunk 0 that the drain also computes.
- Consequence taken deliberately: the short arm is **live code at every peel depth**, not dead at depth 1.

A read hoisted d steps buys latency hiding and owes three things back

Issued naively, every read is followed by a wait for its own completion: the machine stalls the full local-memory latency at each step. Hoisting the read d steps creates three obligations — and each is a **closed form**, never a simulation.

1. Capacity
 $W \geq d + 1$ live generations

2. Prologue
prime steps $0 \dots d-1$

3. Drain
suppress trailing reads

```
dr = off(read, substep) # the REQUEST, per op-class
dr_g = 0 if fan(op) > 1 [veto 1]
    0 if a free sibling is outer to the group's reduction [veto 2]
    max(0, min(dr, W_g - 1)) # the capacity clamp
```

	Shape A ($dr_g \geq 1$)	Shape B ($dr_g = 0$)
read placement	hoisted ahead of its consumer	in place, with a wait
prologue	non-empty	empty
steady advance	non-zero	0

Mixing them is what the `dr_g` refactor exists to prevent — and was the decoder's real state: a Shape-A prologue with a Shape-B steady primes the pipe once and never re-primes it.

Two structural vetoes cut the distance to zero — at any width, for any d

Veto 1 — the broadcast fan. $\text{fan}(p)$ = product of extents of the modes *not* in $\text{pres}(p)$ that are outer, in ord, to **every** mode of $\text{pres}(p)$. If the whole name space is re-walked before any value dies, a hoisted write lands on a name a later consumer still needs — at *any* distance. A race, not an anti-dependency.

ord: outer $\rightarrow$ inner



$\text{pres}(A) = \{M, K\}$: N sits *between* them, so it is not outer to M. $\text{fan}(A) = 1$ — keeps read-ahead

$\text{pres}(B) = \{N, K\}$: M is outer to both. $\text{fan}(B) = \text{ext}(M)$ — $dr_g = 0$

ord	$dr_g(A)$	$dr_g(B)$
KMN, KNM	1	1
MNK, MKN	1	0
NMK, NKM	0	1

Veto 2 — a free sibling outer to the reduction. The sibling is a storage-disjoint group, so the shift cannot carry through it, *and* the prologue's outer factor primes only region 0. Neither half of the pipeline exists; in-place is the only consistent answer.

Both vetoes live in the per-group derivation, not in the min that produces the steady advance — a veto in the min alone zeroes the advance while the prologue keeps priming.

The peel is a vector over levels — and one offset relocates instead of peeling

The retime matrix is indexed by **(op-class, level)**, so *any* level can carry an offset: an outer tile level, the reduction chunk, the substep. PeelDepths therefore reports M_ℓ per level, the per-op-class offsets, the substep request, the derived reach, and every negative offset separately as a deferral.

- PeelDepths.M is deliberately **narrow** — the chunk level, the only one whose prologue/drain the emitter can build — and sits beside peeled_levels(), which reports *all* of them. That pairing turns a silent drop into a loud refusal.
- It used to return an int keyed on the chunk. The tree stayed well formed, the ledger came back empty, and the certificate **certified a schedule nobody requested**.

The cross-level boundary term. An offset at outer level ℓ whose op-class lives strictly *inside* ℓ does not peel ℓ — it relocates the leading δ instances into the previous ℓ -trip's drain. Three conditions decide it, and the third separates these two:

offset	home level	verdict
off(copy, gtile)	iter — an outer, looped level	relocation : whole iterations move
off(copy, iter)	M_split — an inner unrolled axis	ordinary prefetch : the fan stays put

Without condition 3 the two are conflated — the first implementation reported a boundary term for every shipping region-split kernel, and the emitter refuses on a non-empty hoist list, so it refused them all.

The wrap lead has a closed form — and prefetch-1 is degenerate, so three calibrated rules shipped wrong

A staggered workgroup starts at chunk S , walks to $T - 1$, then **wraps** to 0. The compare is against the loop counter — but the counter holds one value per trip, and the increments do not all run in the trip whose number it holds. `lead` is that frame shift.

```
prologue: increment #n targets chunk n, runs at ctr = T, S = T-n => lead = pf - n
steady:   increment M+v+1, runs at ctr = T-v, S = T-n => lead = pf - M - 1
                                     # v cancels: one instruction, all trips
                                     lead = pf - min(to_chunk, M + 1)
```

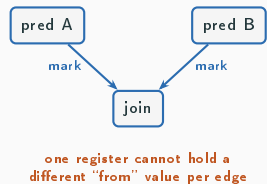
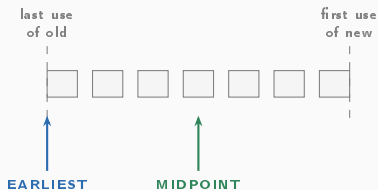
	prologue, to_chunk=1	prologue, to_chunk=2	steady
global prefetch 1 ($M=1$)	+1	0	0
global prefetch 2 ($M=2$)	+1	0	-1

At $M=1$ prologue and steady **coincide at 0**, so any rule fitted there says nothing about $M=2$. Three shipped in sequence — $M - \text{to_chunk}$; 1 if prologue; 1 if $\text{to_chunk}==1$ — each passed its prefetch-1 tests, each gave the prefetch-2 steady increment 0 instead of -1 : the wrap fires early, a whole tensor stride is subtracted, out of bounds.

The same degeneracy bit `copy_must_be_first` and the shared ring depth. Any rule over peel depth, prefetch depth or counter frame must be pinned against *two* prefetch depths in one table. A negative lead is correct, not a sign error.

A Mark has a legal range, not a point — so the policy travels with the region

Two registers are *mutated*, not re-derived: the shared read pointer (a ring swap) and the bulk-copy descriptor. Lowering is split three ways — **analysis** emits pending marks over opaque anchors, **placement** picks a point in the window, **apply** is the only IR mutator.



- **Descriptor changes go earliest** — `gr_inc` and the copy-hop swap, one rule: they mutate one resource and feed one consumer. Every cycle before the load is overlap, and the window's lower bound already *is* the last use of the old value.
- **The read swap goes at the midpoint** — bounded by two *different* resources, so either edge stalls against that edge's consumer. Honestly a heuristic: a bisection of the node list, not of a cost model.
- **One Mark per incoming path** — an exit def in each predecessor, never one at the join. From an AVAIL/ANTIC fixpoint; **TOP is reported, never resolved.**

The degenerate arm is the unpipelined emission — three displacements it must shed

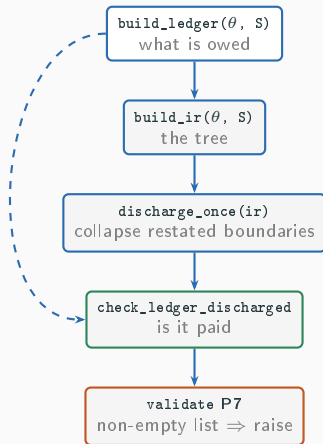
Not “the steady body with knobs turned down”. A steady body assumes a *predecessor trip* supplied its opening producers; a fully-peeled level has neither predecessor nor prologue.

Within the arm, every obligation's producer precedes its consumer.

displacement	mechanism	removed by
retime-carried	the staggered copy ramp; the substep read-ahead	a copy phase with no ramp, and zero read shift — a <i>consequence</i> of emitting unpipelined, not the mechanism
rotation-carried	modulo- <i>S</i> rotation writes each read after the matmul that vacates it; at $W=1$ this is the only pipeline, and no retime value expresses it	not applying the issue-order refinement, whose deferrals both order for a <i>successor trip</i> the arm does not have
order-carried	the copy-first set derived from the <i>steady</i> chunk crossing	re-deriving it: at zero retime a step's own copy is its true producer, so it must lead

Fixing one leaves the other two — three separately diagnosed defects. The ledger caught none: a true-dependency obligation is generation-blind, so `await A@shared` was discharged by *A*'s copy of chunk *t* while the read wanted *t*+1. Empty ledger, passing gate, a buffer nothing had filled — hence a structural test over the arm's own instructions.

The correctness argument is a list written *before* any code exists



An obligation is a hazard, named, so it can be tracked to a discharge.

Obligation(producer, consumer, kind, counter)

- ends are structured Endpoint records — op-class, storage/site, role
- counter is a **completion class** (C_copy, C_read), a model label, not a hardware counter
- **symbolic per op-class**, not per iteration: one row for “A’s global→shared feeds A’s shared→register”

The ledger is small enough to be built and checked on *every* decode.

A closed vocabulary raises on a new kind; a suffix test *schedules* it

RAW_KINDS — true dependency

RAW-residency	consumer needs the moved value
crossing-RAW	read-ahead reaches a chunk <i>this</i> trip's own copy fills

WAR_KINDS — anti-dependency

rotation-WAR	refill writes a <i>different</i> slot
inplace-WAR	refill writes the same slot

OBLIGATION_KINDS = RAW_KINDS | WAR_KINDS,
enumerated in exactly one place. `is_war` is **total over a checked set** — it raises rather than defaulting; `is_raw` inherits the raise.

The prior form was `kind.endswith("WAR")` at eight sites.

It fails in *both* directions as soon as the vocabulary grows:

- a new anti-dependency not spelled ...WAR joins the RAW arm
- a true dependency that happens to end in those letters — read-after-WAR — joins the anti-dependency arm

Neither is an error. **Both are a schedule** — a slower kernel or a wrong answer, found on hardware.

Already paid for once. The suffix match forced *copy-last* on crossing-RAW obligations that needed *copy-first*.

A wait cannot repair an anti-dependency — the read you need has not issued yet

A completion wait makes an instruction wait for **already-issued** operations to finish.

Ring depth S_{shared} , copy running δ chunks ahead. At $S_{\text{shared}} = \delta$ the copy of chunk $c + \delta$ lands in slot $c \bmod S_{\text{shared}}$ — **the slot this trip is reading**.

Ask what the copy could wait *on* in a copy; reads body: the reads that must precede it *have not issued*. There is nothing to name. The only repair is to **move the copy**.

The same asymmetry one ring down: the wmma that must precede a hoisted register refill has not issued either.

Mechanism	Discharges	Enforced by
a wait (an <code>Await</code> naming the producer)	RAW-residency, crossing-RAW	arm (a) coverage; order otherwise free
a position (the issue order σ_c)	rotation-WAR, inplace-WAR	arms (b), (b2), (b3); no wait exists
mere presence (a read the prologue must contain)	readahead-residency	arm (c), against the prologue body

Presence is not a wait and not an order. Arm (c) collects the *coordinates* of every register read the prologue emits and requires each obligation's coordinate to be there. A peel built on the wrong reduction axis is rejected **at build time** rather than by a passing-or-failing hardware run.

That is also why the fifth kind sits outside the closed set — it never becomes an `Await`, so `is_war` must never see it.

“The ledger is empty” means coverage *and* position — and nothing about addresses

Two statements per obligation:

1. **Coverage** — somewhere in the tree an `Await` names the obligation’s completion class. Otherwise it is unsatisfiable however the code is ordered.
2. **Position** — within one straight-line trip, the emitted order of producer and consumer is the one the hazard demands.

Position is not optional *because the backend is order-driven*: it discards the markers and re-derives synchronization from def-use order of the memory tokens. Coverage alone passes an instruction encoding **the inverse dependency**.

What an empty ledger does NOT prove

- It never looks at an address. No arm computes a byte offset, a lane id, a swizzle or an LDS base. “The read reads the bytes the copy wrote” is *assumed*. A wrong-address miscompile is invisible here.
- Generation-blind on the RAW side. `await A@shared` is discharged by *any* copy of A; the wrong-generation case is caught by the separate crossing-RAW row.
- It does not check the ledger is complete. A hazard with no row reports success. **Every historical defect in this file is of that shape: a missing row, not a mis-checked one.**

The cross-check for the missing-row mode re-derives the shared-memory edge set from the lowered CFG and asserts the two sets are equal.

Copies-last is *scoped*, because at one setting the crossing read needs this trip's copy

One steady trip at chunk c **reads** $c \dots c+r$ and **writes** $c+\delta$, where r is the read-ahead's chunk reach.

relation	hazard	order
$\delta = r \ (r \geq 1)$	crossing-RAW	copy first
$\delta \neq r$	*-WAR or none	copy last

Both shipping prefetch points sit at $S_{\text{shared}}=2$, $r=1$ and land on *opposite* sides:

depth 1: $\delta=1$, slot $(c+1) \bmod 2 \rightarrow$ hits the CROSSING read
depth 2: $\delta=2$, slot $c \bmod 2 \rightarrow$ hits the CURRENT read

An earlier predicate was $S_{\text{shared}} > r$ — true for *both*, so it emitted the depth-2 point copy-first and overwrote the buffer the trip was consuming.

The requirement is scoped to $S_{\text{shared}} \leq \delta$ — there the refill is **in place** and the order is mandatory. Otherwise it is a rotation and the order is free. Both order checkers key on in-place-WAR *alone*: a rotation-WAR copy emitted first is not a violation.

The emitted policy is broader: σ_c defers *both* WAR kinds unless vetoed — “last unless it must lead”.

The veto is only checkable because the RAW was modelled too. With only the WAR present, nothing forced a copy to precede its own consumers — correct at one chunk, wrong from two up.

A hazard analysis finds conflicts between things *present*; absence is the schedule's job

Hazard analysis (layer 2)
walks the program, pairs the shared
accesses it *finds*, classifies RAW
/ WAR / WAW, places a cover

Schedule model (layer 1)
knows which reads the prologue
must contain, because it de-
rived the peel that warms them

A read that was never emitted collides with nothing. It raises no pair, mints no edge, leaves no obligation undischarged.

Measured, twice.

- A drain read-ahead suppression window widened by one step: bf16 NT, $16 \times 16 \times 32$, 2×2 wave tile, DepthU 64 — **62 ds_reads at prefetch depths 1–2, 54 at depth 3**, against an unchanged 36 matmuls. Eight local reads gone, **empty ledger, no warning**.
- The short-arm fold's UNSOUND verdict: an arm that reads a register it never wrote. Repaired for the data operands, **never covered the microscaling scale operands**.

So existence is checked separately, and structurally.

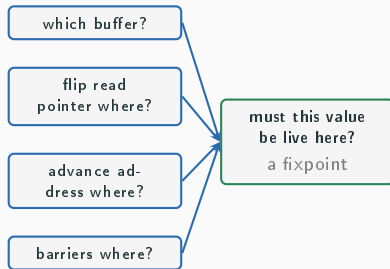
- readahead-residency is an *existence* obligation: arm (c) demands the coordinate be in the prologue body.
- Closed-form gates
(*chunk_crossing_violations*) prune bad (*dr*, δ) pairs *before* emission — far better as a message: “operand A, read-ahead 1, reach 1, depth 1, needs 2”.

Four physical questions — and all four are the same question

1. Which of the S rotating LDS buffers does *this* access touch?
2. At which instruction does the LDS **read pointer** flip to the next buffer?
3. At which instruction does the **global read address** advance to the next reduction chunk?
4. Where do **barriers** go?

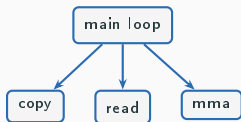
Each one reads: *one physical register or resource must hold the right value at this program point.*

That is a **compiler dataflow problem**, not a scheduling problem — and the scheduler is the wrong layer to ask.



A rolled tree has no point to reach and no value to propagate

LoopIR — layer 1



```
buf = (k+2) mod 2
```

rolled: one node per *kind*,
not per dynamic instance

GIR — layer 2



```
g =  $\phi(0, g')$   
g' = (g+adv) % S
```

unrolled: stages are real blocks,
each generation a named SSA value

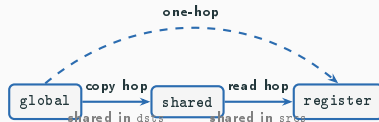
Unroll the pipeline stages into basic blocks, give every buffer generation a ϕ at the loop header — and all four questions become **textbook fixpoints**. *That* is the reason for two IRs, not layering for its own sake.

The node set is semantic: a swap is a fact, never an `s_xor`

group	nodes	answers
nouns	Tile, Gen, Ref	what data / which buffer generation / one use of it
verbs	Move, Mma	one movement hop / one multiply-accumulate
point	Mark	a semantic fact at a program point
control	Goto, CondGoto, LoopBack, Return, CondChain	the CFG terminators
graph	Block, GenPhi, GenXfer, Program	the CFG itself

- **No physical fields** (checked by G-SEMANTIC): no address, no VGPR colour, no byte offset. The same fact realizes as a descriptor `s_xor`, an address `v_xor`, or nothing at all — a node naming one cannot be reused by the others.
- **One SSA value class**: Gen, the loop-carried buffer generation. Registers are *not* SSA — their rotation width is a schedule decision this layer only validates.
- **Six Mark kinds**, closed set — and **no barrier kind**: GIR carries the ordering obligation, layer 3 picks `s_barrier` / a wait count / nothing. (The comment explaining this is one rename stale — it still says await.)

A hop is derived from the spaces, and a Ref names four residencies



One Move class, no Copy/Read subclass — so the analyses can walk every shared touch without a type switch, and "which movement is this?" is a **derived** property every new consumer must be told to get from `refs.py`.

Ref field group	residence
gen, gdelta	loop-carried shared
abs_gen	pinned shared (peel, short arm)
group, slot, reg_ring	register
covers	movement-quantum footprint

gen vs **abs_gen** is the peel/steady distinction made structural: a prologue fill knows its buffer at compile time, a steady access does not.

"region" means **two unrelated things** — a *storage* region (one piece of a split LDS tile) and a *placement* region (the legal window for a Mark). They never share an expression, but they do share a file list.

An insertion has a legal *range*, not a correct point



- EARLIEST descriptor edits
- MIDPOINT LDS read pointer

- Anchors are **node references**, not indices — a window survives other Marks landing in the same body before it resolves.
- Policy travels *with* the region: a descriptor edit wants every cycle of overlap it can get; the read pointer is bounded by two different resources, so neither edge is right.
- Baking the index into the analysis is what the first version did — and it is why placement policy could not be changed without rewriting the analysis.

One typed exception: `StructuralPass` may reshape the CFG. It exists because `ApplyMarksPass` called itself "the only mutator" long after the fold pass had started **deleting blocks**. What is still **unchecked**: two analyses placing conflicting Marks in one window.

The region is an AGENT fact on the read hop, a COORDINATE fact on the copy hop

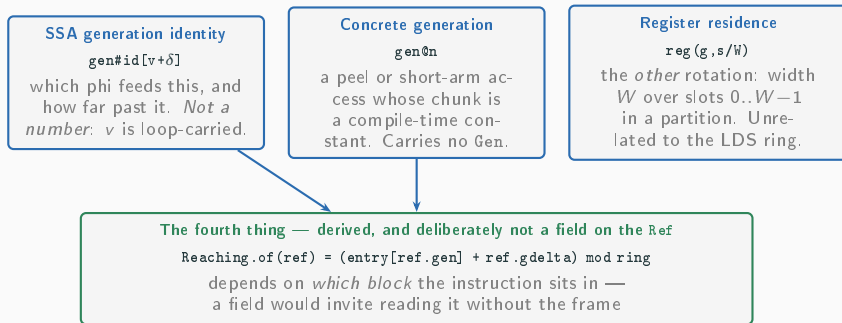


A dump asserted, for a while: "wave *w* reads region *w*, so one agent's reads name one region." Both halves false — the block listing printed directly beneath it showed A reading *both* halves, and 8 of 8 agent-relative operands had disagreeing hops.

consumer	what it does with the same one fact
mem_tokens (ordering)	widen : the read names <i>every</i> region's token, or the producers it did not name go unordered
check_region_coverage	narrow the demand : under-coverage exempt (one agent never covers the set); over-coverage still faults — a partition member cannot be a non-member
render.py (the dump)	print the per-hop split, and retract the sentence above

Three checkers each had to be taught this independently — one concept that should have lived in one place. It also cost a day: a coverage message was read as proof the *model* needed a new narrowing construct, when the checker was asking a coordinate question about an agent fact. A violation message is evidence about the checker's premises too.

A residence says three different things — and the number every consumer wants is a fourth



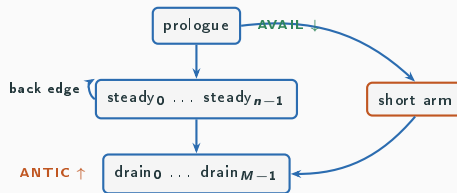
GenReaching domain: per block, $\text{Gen} \rightarrow \text{int}$, a flat constant-propagation lattice · direction: forward · meet: *agreement-or-bottom* (a Gen one predecessor supplies and another does not is **bottom**, not inherited).

The dump printed (1) as `gen0[v]` and (2) as `gen=0` — one character apart — and chained the three with early returns, so a register residence was invisible whenever a generation was present.

The pointer is one register and the program is one execution — so it is ONE timeline, not three regions

Three seam defects, all shipped, all silent:

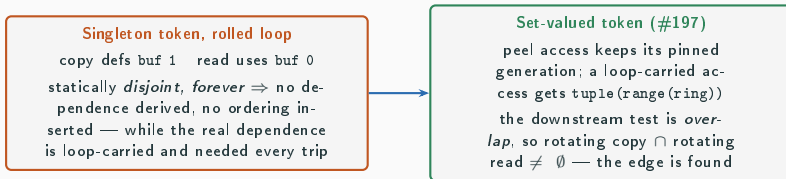
1. **Double swap.** A per-block (need_in, need_out) summary *plus* a hand-written per-edge-type table. Position and value computed by different code; nothing forced them to agree.
2. **Wrong drain frame.** Drain requirements are offsets from T , steady ones from the current trip. Compared as raw integers, so a change of coordinate system was “reconciled” with a swap the hardware does not need.
3. **No increment at all.** “One Mark per copy in a loop header carrying a back edge”: with a multi-block steady body the two filters are mutually exclusive $\Rightarrow$ zero increments, every trip re-fetching the same tiles.



Formulation: a requirement is a value demanded at a point; a swap / increment is a *definition* of that value; “where do defs go” is classical code motion — forward **AVAIL** $\times$ backward **ANTIC**, a def wherever supply meets a different demand. Crossing $p \rightarrow b$ re-expresses the value via `edge_delta(p,b)`, so a drain value and a steady value are never compared as raw integers. **Three pointers (LDS read, TDM descriptor, global-read address) share one 250-line solver; both `on_conflict` callbacks *raise* — report, never resolve.**

A token names a SET of generations — that is exactly what makes a loop-carried dependence visible to a backend that only sees an order

A memory token is not a label: it is an **LDS pseudo-register**. Downstream, a copy *defs* it, a read *uses* it, a barrier does *both* — and ordinary def-use machinery does the rest. The token relation is an alias relation and nothing else.



The asymmetry that governs every choice here

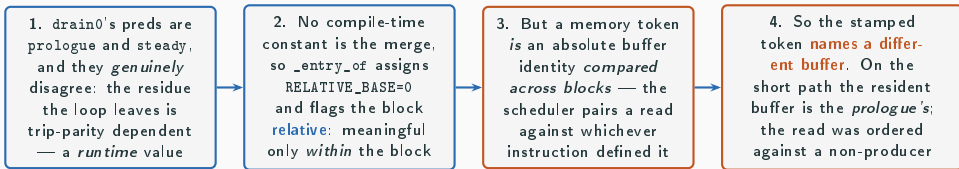
token too narrow	a miscompile
token too wide	a redundant wait, never wrong

So every deliberate imprecision is a *widening*.

And then the set was retired (#207)

A set is a *may*-set on a channel written for a *must*-set — and it says *that* two instructions collide, not *after how many trips*. Once GIR derived the hazard edges itself and stood a **fence** between the ends, the fence carries the must-set and receives each token as def *and* use, so copy $\rightarrow$ fence $\rightarrow$ read chains directly. Tokens went back to naming one buffer.

If the token is not trip-invariant, the derived wait guards the wrong producer — measured as -nan



The measurement. On a microscaling fp8 kernel: -nan in 23 of 540 swept cells, *every one on the short path*. A wrong-generation E8M0 scale poisons the multiply — wrong-generation *data* often stays finite and silently wrong.

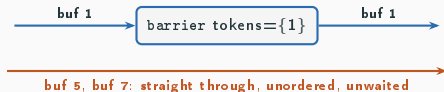
The fix, in the module's own idiom: in a relative-frame block the access may touch every generation of the ring, so it carries every id. A redundant wait, where being optimistic loses one.

Two edges to keep: an `abs_gen` ref widens to `range(1)` — correctly, it names a concrete buffer whatever its block's phi frame does. And relativity must stay *transitive*: `drain1`'s only predecessor is `drain0`, and “one predecessor trivially agrees with itself” once dropped the flag.

Still open: a *legitimate* representative-trip frame read as invariant by a cross-trip consumer. Masked in multi-wave (a fence stands between the ends anyway); single-wave has no fence, so the token is the only carrier. *Before narrowing a token, ask which consumer reads it.*

Barriers are PERMEABLE PER TOKEN — so a fence must name everything live across it

A barrier is a *point*, and its ordering power comes entirely from its token set: the downstream pass gives it each token as both a def and a use. **A buffer the barrier does not name is not chained through it at all** — accesses to it migrate across freely, and get no wait either.



The design rule that follows: it does not get to be minimal in both dimensions. The set of **POINTS** is minimised (a greedy set cover over admissible slots); the set of **NAMES** on each point is *not*.

The measurement that proved it. A union-over-covered-edges fence was propped up by an unrelated over-approximation. Fixing that widening shrank one steady fence's token list from (1, 5, 7) to (1,) and took a multi-wave split configuration from 42 failures to 62. The *placement never moved* — only the names on it did. The widening had been doing liveness's job by accident for months.

Ownership boundary

This layer owns: the memory **token**, and the **copy order**.

It never owns: `s_barrier`, `vmcnt/tensorcnt`, wait strength. There is no barrier kind in `MARK_KINDS` and never was — the kind is fence.

One obligation, three realizations (barrier / wait-count / nothing). Name one and you are discarded on the path that wants another.

Existential reachability is not betweenness — so ask the separation question directly

The cover is computed per block, so without a cross-block check it is blind: the last steady copy feeds reads in *two* drain blocks, and the second asks for a fence the first already provides. “Is a fence in an earlier block already on every path?” was approximated as:

```
x in reachable(producer_block)    and    x in dominators(consumer_block)    - THE OLD FORM
```

Why it is wrong: reachable is *existential* — “some path from the producer reaches x”. A candidate reachable only along a **back edge** satisfies it while sitting *before* the producer on every forward path. The hand-written `x != pb` guard was one instance of that general failure, seen once.

```
_separates(x, pb, cb, succ):  
    if pb == cb: return False  
    stack = [s for s in succ[pb] if s != x]  
    while stack:  
        lab = pop(stack)  
        if lab == cb: return False # path avoids x  
        if seen or lab == x: continue  
        stack += [s for s in succ[lab] if s != x]  
    return True # every path forced
```

Starting at pb’s *successors*, not at pb, is what makes a fence in the producer’s own block fail correctly.

Delete x from the graph. Is the consumer still reachable from the producer? If not, every producer→consumer path is forced through x. One predicate subsumes reachability, dominance *and* the guard — and needs no dominator tree.

Measured: the old form left **248 unfenced producer→consumer paths in 14 of 36** swept configurations, all `steadyn-1:copy → drain0:read`, all at multi-block steady bodies. The separator test drives **248 → 0** for **+1 fence** in 16 configurations — and every single-block-steady configuration, which is everything shipped today, is **bit-identical**.

An LDS hazard is a trip distance — and exactly one bit of it is load-bearing

An access carries offset g (g_{delta}) on a ring of depth S ; the back edge advances by adv :

$$\text{buf}(v, g) = (\text{base} + v \cdot \text{adv} + g) \bmod S$$

Two accesses collide when

$$(v_c - v_p) \cdot \text{adv} \equiv (g_p - g_c) \pmod{S}$$

and with $\text{adv} = 1$ (every kernel today) the **trip distance** is

$$d = (g_p - g_c) \bmod S$$

The base cancels — so no absolute buffer identity is needed, which is fortunate, because in the one place it would be needed it provably does not exist. Deriving d from resolved generations instead would compare *representative-trip* values and under-report every pair that agrees on a different trip.

Why the back end cannot do this: its phase machine walks a flat stream keyed on a static label. It cannot see a dependence whose producer and consumer are the *same static instruction on two different trips* — which, in a rolled loop, is every loop-carried LDS dependence.

family	dir.	what breaks without ordering
RAW residency	$w \rightarrow r$	read sees stale data
WAR rotation	$r \rightarrow w$	copy overwrites unread buffer
WAW rotation	$w \rightarrow w$	two copies, unconstrained order

The one bit: $\text{same_trip} \equiv (d = 0)$. $d = 0 \Rightarrow$ discharge *inside* the body, between the two accesses (an *interval* of admissible slots). $d \neq 0 \Rightarrow$ purely loop-carried, so any fence in the body is crossed on the way — a *union of two intervals*. **That union is why fence placement is a set cover and not a shortest-interval problem.**

Two independent derivations agree: TensileLite's barrier pass skips at `numWaves == 1`; this analysis reads only per-op-class agent distribution and finds 84 hazards on each of three single-wave fixtures, of which **zero** need a fence.

One read *act* is N instructions spaced by the LANE_INTERLEAVE — not by the per-thread input count

A read Move names one (tile, reduction substep) act. Layer 3 realizes it as N ds_reads, and both N and the *spacing* are geometry GIR never sees. The natural guess — “this lane’s elements are one contiguous run” — is wrong in the spacing: consecutive chunks are **LANE_INTERLEAVE** chunk-widths apart, because the other lanes covering that K range each take a chunk in between.

$$\text{unrollElems}(i, v) = (v + i \cdot n_{\text{in}} \cdot \text{LANE_INTERLEAVE}) \cdot \text{vwTrLoad} \qquad \text{LANE_INTERLEAVE} = 2 = \frac{\text{WavefrontSize}}{\text{matrixInstTO}}$$

shape	bw	bpe	lrwv	ipt	miK	n_{in}	n_{out}
bf16 LDS-tr	4	2	8	16	32	1	2
bf16 general	4	2	8	16	32	1	2
fp8 LDS-tr	2	1	16	32	64	2	2
fp8 general	4	1	16	64	128	1	4

Outer spacing = $n_{\text{in}} \cdot \text{LANE_INTERLEAVE} \cdot \text{vwTrLoad}$; the guess is `inputPerThUnroll`. They agree only when the chunk count equals the interleave — *every* bf16 shape and *no* fp8 one.

bf16: $1 \cdot 2 \cdot 8 = 16$ vs `ipt` = 16 — identical. The wrong rule passed every test for as long as bf16 was the only data type.

fp8 general:

```
correct ((0,0), (32,4), (64,8), (96,12))
guess   ((0,0), (64,4), (128,8), (192,12))
```

Three of four reads at the wrong K position. **The registers still tile perfectly**, so a check that only verified “no gap, no overlap in registers” waves the wrong table through. Hence two independent checks, and a refusal rather than a plausible answer: **REGISTERS and K EXTENT** ($n_{\text{out}} n_{\text{in}} \text{vwTrLoad} \cdot \text{LI} = \text{MatrixInstK}$).

Consequence for hazards: `Access.pos` is a position among *acts*, not instructions. A fence between two acts is between all N fragments of each — true for free, but only because a fence is never placed *inside* an act.

The short-path fold is a reordering-legality question — and it is decided on VALUES reached, not producer identity

The degenerate ($T \leq M$) path exists in every pipelined kernel. The scaffold serves it by *skipping* the loop; LoopIR emits a disjoint unpipelined arm. Folding deletes the arm and retargets the guard’s false edge into the drain head — legal only if every def-use pairing survives the reordering (copy,compute,copy,compute $\rightarrow$ copy,copy | compute,compute).

Why producer identity gives confident false SPLITS. The two arms are *deliberately* different schedules: the arm never enters the loop, so its read-ahead is reset ($dr = 0$) while the drain keeps $dr(\theta)$. They are not instruction-isomorphic, so “same instruction” is not a well-defined question.

So each producer carries a value: a copy is (‘copy’,ops,coord,occurrence) — the k -th copy fetches chunk k on both arms — and a read chains **transitively** to the copy that filled its buffer. Naming the *generation* instead identifies the chunk only while $ring \geq peel$: at $S = 1$ every generation is 0, a false FOLD on exactly the configuration that needs the answer.

Under producer identity the shipping default reported 24 breaks — every one an artefact.

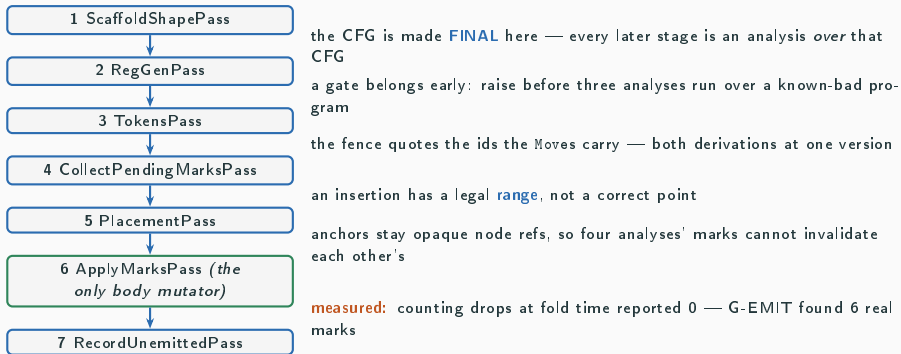
sweep	verdicts
A: 6 orders $\times$ PGR 1–3 $\times$ PLR 0–1, $S=2$ $M=1, 2$ fold; $M=3$ splits, 8 overwritten	24 FOLD / 12 SPLIT
B: same shape forced to $S=1$ 8 and 16 breaks, all overwritten	$M=1$ folds; $M=2, 3$ splits
C: 22 named fixtures	19 FOLD / 2 SPLIT / 1

Loop order does not enter the answer in any cell. The two C splits are *not* ring-depth splits: 32 breaks over 4 register locations, all unwritten — the register-side warm-up gap.

The summary is “fold iff $ring \geq peel$ ” — and the analysis deliberately does not test that inequality. It sees nothing of warm-up (an arm reading what it never wrote is UNSOUND, and the inequality calls it foldable).

The shipping default folds in all six orders — as it must, since the scaffold has done exactly that for years.

The pass order is not a preference: every adjacent pair is fixed by a dependency



Then two gates: `verify_gir` (twelve G-* checks) and `_check_rotation_waw`, the one invariant only true of a *finished* program.

Adding a pass: the slot in the order is a claim you must justify

1. Kind. Are the block set, terminators and every preds/succs identical on the way out? Yes → Pass; no → StructuralPass.
2. Slot.
 - touches the CFG → inside `ScaffoldShapePass`, never later
 - produces `PendingMarks` → not a pass at all: an `Analysis` in `CollectPendingMarks`' list
 - records a fact about the finished program → after `ApplyMarks`
3. CFG changed → extend the verifier (next slide).
4. Test three ways: standalone, through the pipeline, and a **negative control** that builds the illegal shape.
5. Return what you invalidated — and know that nobody reads it.
Rely on `prog.bump()`.

```
class MyPass(Pass): # or StructuralPass
    """What it changes; WHY this slot."""
    def run(self, prog, am):
        f = am.get(SomeAnalysis(), prog)
        ...
        if changed:
            prog.bump() # MANDATORY
        return ("invalidated",) if changed else ()
```

Rules a reviewer looks for:

- key on `Block.phase` or CFG shape, **never a label**
- never re-derive a θ fact — look it up in `prog.meta`
- rebuild succs with the terminator; recompute preds wholesale
- guard idempotence: tests run passes standalone
- **report, never resolve** — raise naming every instance and the config

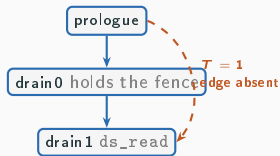
A checker that reports clean because it could not tell is worse than no checker

Two checkers, disjoint failure modes: `verify_gir` asks *is this IR well-formed?* (raises on the first violation); `check_plan` asks *does this schedule compute the right GEMM?* over the flat emit plan (returns a list). Neither subsumes the other.

	what it proves	how it can still pass (named in the source)
G-TERM	preds/succs agree with the terminators	an <i>empty</i> tuple is never checked — opt-in per block
G-CFG	exactly M drains; 0 <i>or</i> exactly M short blocks, agreeing with the verdict	<code>meta['M']</code> absent disables the count entirely
G-WALK	the region walk closes <i>and</i> every copy is on the region the descriptor is on	a block with no split copies is trivially clean
G-TOKEN	disjoint LDS storage never shares an id; the <i>stamp</i> matches the analysis	the fence cover is checked non-empty, never <i>sufficient</i>
G-EMIT	marks dropped into blocks nobody emits are counted	compares a count, not the kind multiset
G-UNIFORM	a proc-scoped rendezvous is on every agent's path	structurally inert: role is 'all' everywhere and <code>Pred</code> cannot spell an agent test

The guard: every check needs a negative control — a fixture that breaks the property and asserts the checker fires. `walk_violations` first checked only the *net* displacement; hoisting every increment mark to the block entry leaves the net at zero while every copy loads the wrong region. Net-clean and uniformly wrong. Its control now parameterises over `drop/duplicate/hoist`.

A pass that changes the CFG obliges the verifier to CHECK the new shape, not assume it



With the edge missing, *drain0 dominated drain1*, so the fence cover elided *drain1's fence*. True of the CFG it was given; false of the kernel.

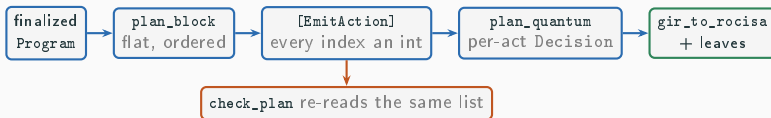
Measured: 32 cross-agent hazard edges left unfenced; the microscaled-fp8 kernels returned -nan on every $T \leq M$ cell. The repair was structural — all three CFG rewrites moved into one stage that runs **first**, so “the CFG is final before anything reasons over it” is a property of the pipeline's shape, not a convention.

The verifier side of the same rule:

- **G-CFG** the fold may leave **0** short blocks (folded) or **exactly M** (kept) — **any other count** means a pass truncated the arm. Before the cross-check, silent truncation was indistinguishable from a correct fold.
- **G-EMIT** a block nobody emits must *say so*, and its discarded marks must be counted.
- **G-TRIP** a pass that rotates the loop cannot quietly change the total work.

Worked example — a critical-edge splitter: `StructuralPass`, after the early exits, before labelling. Its new block phased “`drain0.split`” would be *counted as a drain* and fail `n_drain != M`. phase is a string doing type-system work.

Load versus broadcast is DERIVED from the movement quantum, never declared

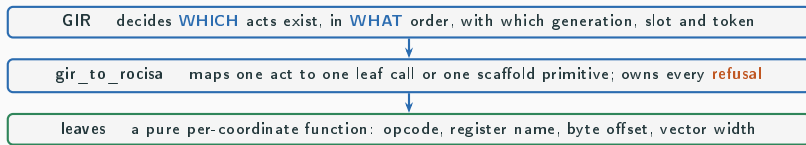


A quantum is the set of axes one movement *instruction* covers, supplied from outside the model as two index expressions over the tile coordinate. Nothing is tagged a broadcast:

mechanism	carrier(t)	slot(t)	regs
wide load of 2 tiles	$t/2$	$t \bmod 2$	2 — each tile gets its own register
sub-agent broadcast, N partitions	$(t/N) \cdot N$	0	1 — partners SHARE a slot

The group is the **preimage** of carrier, not a contiguous run — the sub-agent partner sits a stride away. **Measured:** a leaf that merged two acts on its own authority folded LDS tokens 9 and 8 under one ds_load_b64; 4096 of 65536 outputs wrong. And WHOLE = None is *silence*, not a permissive decision: returning Decision(emit=True) instead made the leaf skip its own fold and failed 16 of 16 microscaling cells, including the 10 that had been passing.

The leaf boundary decides HOW — never WHAT, WHERE or WHY



- **The claim that licenses it:** for a dense WMMA GEMM the machine code for one tile does not depend on the loop order at all. The schedule decides only which coordinates exist and in what sequence.
- **Nothing is re-derived.** Every fact arrives on the act or from a call into the active `KernelWriterAssembly`. Register names come from `generateSrcStrForMFMA`, accumulators from `accVgprReadWriteIndex` — because re-spelling them here is a second derivation that agrees on the common case.
- **Refusals live in the adapter**, not in the leaves: six `NotImplementedErrors` meaning *GIR can express this, the scaffold cannot realize it*. A shape whose formula was never compared against the scaffold's own output is rejected by name, not generalized.
- **The boundary is sharp and it has been crossed.** `Decision.reg_slot` is the *carrier* (the instruction ordinal), not the *slot* (the position inside it). Feeding `slot` to the leaf renamed every scale register and failed all 16 microscaling cells; the test that certified it had asserted the wrong one of the two.
- **Honest residue:** `Decision.narrowed` is a socket nothing wires — narrowing needs an instruction-selection choice the caller owns. Either wire it or delete it.

Two files import nothing — and that emptiness is the feature

`tdm_split.py` imports `dataclasses`.

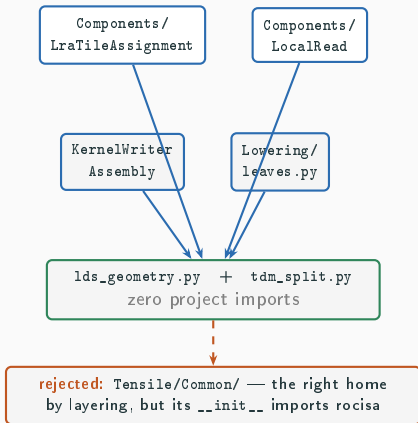
`lds_geometry.py` imports one name from `tdm_split`.

No `Solution`, no `KernelWriter`, no `tP`, no `rocisa`.

Functions needing more than three scalars take a **duck-typed context** — so the *declaration* can live at the caller and the *arithmetic* lives here.

Consequence: `region_row_elems(32,0,False,2)` can be called in a bare `pytest` and `asserted == 16`. The test it replaced had to match *source text*, and broke the moment the wrong expression was corrected.

Do not add an import to either file. Both docstrings state that obligation.



Four emitters, three layers, **one derivation**. `Components /` reaches *up* into `Lowering /` on purpose. The one remaining option — duplicating the formula — is what caused the defect two slides on.

A partition packs blocks iff it cuts the image's *inner* axis — so the rule is a diagonal

LDS is linear, so the tile is serialised in one of two orders, chosen per operand by the **memory layout** of the global tensor:

unroll-major	image [free] [unroll]
tile-major	image [unroll] [free]

The hardware writes each region **densely**. Cut the *outer* axis and the regions are runs of whole rows, adjacent — the unsplit image is reproduced exactly. Cut the *inner* axis and a dense write cannot interleave stripes, so what lands is a different image.

Stated once, in `tdm_split.split_packs_lds`:

```
| return (axis == AXIS_MT) == (not unrollMajor)
```

an XNOR of "the cut is on the free axis" with "the free axis is inner"

cut axis	unroll-major [free][unroll]	tile-major [unroll][free]
AXIS_MT free	contiguous	PACKED
AXIS_DU unroll	PACKED	contiguous

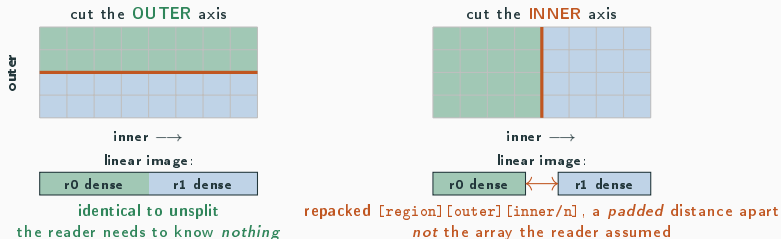
Both cells of the MT row are measured — one in the negative.

MT, tile-major → PACKED (2026-08-17, NT MT32x32x64): without the packed treatment every NT split kernel read region 0's bytes for half its tiles.

MT, unroll-major → CONTIGUOUS (2026-08-16): *adding* a region term here regressed 82 passing kernels, 110 failures → 192.

Same axis, two layouts, opposite treatment. **Generalising either measurement to the other layout is the defect.**

A packed split makes the address PIECEWISE — and every consumer pays for it



The packed reader owes **three** corrections, and they must all come off the *same* predicate: a per-region **block base**, a **shortened row** ($l/n + \text{pad}$), and an **in-region inner coordinate** ($\text{inner} \bmod l/n$).

A division is unavoidable. $\text{region} * \text{splitBoundary}$ is a *padded byte* boundary, not an element stride, so the piecewise form has no linear expression. Any emitter that accumulates $(\text{rIdx} * \text{step} + \text{localReadOffset})$ must divide — `fold_inner_offset` does it once so the scaffold path and the GIR path cannot disagree about where the boundary is. Straddling reads are excluded by a `Solution.py` rejection, $(\text{DepthU} // n) \% \text{MatrixInstK} == 0$.

Every split failure traced to *one* missing term in the tile stride — not tokens, not waits

Turning splits on under GIR failed on hundreds of kernels, on both layouts, at several loop orders. It had every characteristic of a **synchronisation** bug: depth-dependent, changed with prefetch, different at one wave vs four.

Memory tokens, derived waits, barrier placement, hop order — **all of them were correct.**

```
tile_row(t):  
    was: tileStrideElems * t  
    is : (t // VW) * MIWaveGroupShape + (t % VW)
```

a wave takes VW adjacent rows, then jumps a whole group — that jump is what hands the intervening rows to the other waves

Measured (MT32x32 KMN PGR2/PLR0, *unsplit*), operand A ds_load offsets:

```
scaffold: 0,32,64,96, 2176,2208,2240,2272  
ours : 0,32,64,96, 128, 160, 192, 224
```

Wrong rows → sometimes the wrong *region* → the *correct* token machinery then ordered a read against a producer that was not its producer. **The symptom presented one layer above the defect.**

Three reasons it stayed invisible

1. **The wrong form is a special case of the right one.** At `MIWaveTile == VW` the group term is zero — and every fixture pinned `MIWaveTile == VW == 2`.
2. **The failure was numeric, not structural.** `ds_read offset:128` is as well-formed as `offset:2176`.
3. **The address is a sum of independently-chosen terms.** One wrong term still yields a *plausible* address — right magnitude, alignment, region count.

1. **ADDRESS ARITHMETIC** —
is every term in the same frame?



2. **REGION / TOKEN LABELS** —
does the label match the address?



3. **ORDERING** — waits, barriers, hop order

Ordering is generic and covered on every build. **The prior is on the arithmetic.**

A model is verified only when a *second, independently derived* implementation agrees

The LDS address is not computed by `LraTileAssignment` — it is *emitted* by it. Two ways to check it:

(a) re-derive the rules in Python

a transcription: agrees when both are right *and* when both are wrong the same way

(b) interpret the emitted instructions

can only be wrong about *ISA semantics* — a surface where both authors had to agree with a third party

`asm_lane_eval.LaneEval` executes a straight-line slice of `gfx1250 asm` once per lane (32 lanes, `Wave32`) and reports per-lane register values. `lds_region_check` then divides each address by the writer's *own* split boundary and asks: does one `ds_load` touch two regions?

Two distinct faults: **within-wave** straddle — *unrepresentable*, no coordinate fixes it; **across-wave** — representable, but a race if the model was not told.

Refusal is the design

Any opcode or operand it cannot model raises `UnsupportedInstruction`. A silently-skipped instruction leaves a stale register, and the address then *looks like an address*.

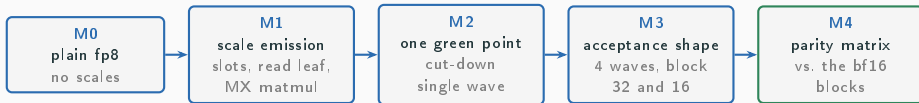
The first version returned 0 for an unwritten VGPR, reasoning that a backward slice is closed under its own producers. **It is not** — it bottoms out at kernel *inputs*. Every address register then evaluated to exactly its offset: immediate, all lanes looked like region 0, and the checker reported **zero violations for four kernels**: a clean bill of health produced entirely by reading uninitialised state.

And the corollary: a differential oracle must first be shown capable of disagreeing at the input in question.

`region_derivation_check` recomputes the region from tile *position* vs tile *index* — it pinned the wave-span defect — but it shares `tiles_per_region`'s `max(1, . . .)` floor, so at `MIWaveTile 1` both derivations collapse to zero and it reports agreement it is **incapable of withholding**.

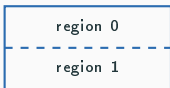
Add an oracle here, and add with it a test that makes the oracle fail.

MX-scaled fp8 reached parity in five staged steps, so every failure had one owner



- Each step adds **exactly one** new fact, so a miscompare after step k has one candidate cause. M2 is deliberately a *cut-down* configuration: one green point before any growth.
- End state: **404 of 404** MX-scaled fp8 kernels and **152 of 152** plain fp8 kernels emit byte-identically against the reference path; the acceptance matrix is now 444 kernels.
- MX is where the **movement quantum** became load-bearing — one hardware load may carry several logical tiles. A `ds_load_b64` folding two scale tiles named only one of its two memory tokens (9 and 8, only 9 named) and **miscompared**.
- Making “no merge supplied” a *permissive* default failed the **entire** microscaled matrix, including the ten cells that already passed: a permissive default is not safe when the fallback *is* the real behaviour.

Tile splitting went per-operand — and which axis the cut falls on decides the LDS image



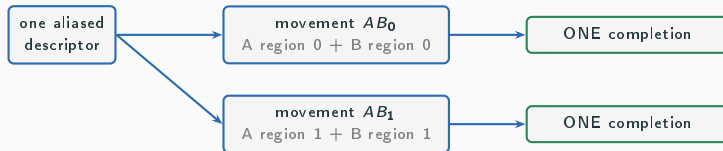
cut the **outer** axis: packed



cut the **inner** axis: interleaved

- TDMSplitA/TDMSplitB are **three-valued per operand**: off, cut the free axis, cut the reduction axis. The knob names an axis; the factor is fixed at 2.
- The schedule model above it is deliberately *more* general — a positional four-list with any factor on either axis — so it can be exercised ahead of the emitter.
- Packed or interleaved follows the diagonal rule of the previous chapter. How it *shipped* is the part worth keeping: the transposed-N layout inverts the rule, and an earlier version **cut the wrong axis** there — the two answers had coincided three times before a layout appeared that separates them.
- Scale, metadata and sparse operands are never split.

A fused copy is one completion, and its members pair by index — nothing else names the pairing



- A is cut on its *free* axis, B on the *reduction* axis. There is no shared mode to name the pairing — **the index is the pairing**.
- Awaiting one member awaits an event the hardware **never signals separately**, so the obligation ledger discharges the **movement**, not the operand.
- The defect: the schedule instance carries *one* coordinate, necessarily the representative member's axis. Every other member's region axis was left unpinned, and widened.
- Measured on a multi-wave transposed-N split, the fused copy's tokens came out (1,5,7) and (3,5,7): A's regions correctly distinct, **B carrying both of its buffers on both region copies**. B's halves became indistinguishable to the hazard analysis.
- Where a member is split on *several* axes the pass **refuses** rather than guessing: an index alone does not say which axis it pairs on.

One barrier per obligation was not a worse analysis — it was an unbatched one

formulation	problem shape	result	verdict
one fence per obligation	—	28–48 barriers; the back end needs 7–10	the baseline
interval greedy	interval cover	splitting a union over-fences	wrong, not just larger
ILP / exact search	set cover	microseconds at this size	unstable tie-breaks
greedy cover	set cover	6 fences for 1284 cross-agent edges	shipped

- A fence is a **cover**, not the realization of an obligation: one point discharges every edge it happens to stand between. Only cross-agent edges need one, so a single-wave kernel gets **zero** — independently agreeing with the back end's own one-wave skip.
- A loop-carried pair's admissible slots are $\{s > a\} \cup \{s \leq b\}$ — a **union wrapping the body**, not an interval. That one fact forces set cover.
- The *points* are minimised; the *names on each point* are not — token permeability, from chapter 6, is what forbids minimising both.
- ILP was rejected on determinism, not cost — solver tie-breaking would make the emitted assembly non-reproducible.

What went wrong: two places computing one fact, agreeing on the common case

the one fact	the two derivations	where they diverged
the region walk	the loader vs. the incrementer	+2 steps against -1: descriptor a chunk ahead
the rotation width	the scheduler's count vs. the name table	the degenerate in-place group
the token set	the analysis vs. the stamp on the move	a pass that truncated the stamp
the read-ahead depth	three different denominators	same on 100% of the matrix, differing in <i>form</i> on 47% of it
the region a tile is in	the name it carries vs. where its lanes sit	agree at wave group [1,1], not at [2,2]

- **A differential probe reporting “no difference” is worthless until it is shown able to report one.**
The tile checker applies the same $\max(1, \cdot)$ floor as the derivation it checks: at one tile per two regions both collapse to region 0, so it is *incapable* of disagreeing. A gate was removed on exactly that argument and the client **segfaulted** — 800 result rows before, 686 after. Reverted, with the argument preserved because it will be made again.
- **A rejection that only appears under the new path is a parity failure, not a safety feature** — and a silent one: the solution stops appearing, its reference twin still passes, the matrix reads green with the arm under test absent. So the gate **raises**, and its contract is that it must never fire.

How it is validated: four layers, and each one is blind to something

layer	what it proves	what it cannot catch
structural checks, on the build path	schedule-gate properties, GIR well-formedness, and a per-kernel semantic check over the emit plan — pure Python	they read the structures the passes wrote, in the same vocabulary: good at inconsistency, blind to the model being uniformly wrong
fixtures and independent oracles	22 named scenarios; a 168-cell order $\times$ split $\times$ prefetch sweep; simulators sharing <i>no code</i> with what they check	coverage bounded by whoever wrote the sweep. One check was correct, had four negative controls, and ran only from tests; a later sweep flagged 216 of 216 single-substep cells and 0 of 432 elsewhere — all sixteen hardware miscompares of that shape
CPU-only generation, emit-diff vs. the old path	152 of 152 fp8 and 404 of 404 MX-scaled fp8 kernels byte-identical; both arms build locally	byte equality proves you changed <i>nothing</i> , so it is silent on any configuration the old path cannot generate; the coarse offset/count diff has missed two defect classes by ignoring registers and order
hardware numerics	the only layer that sees what the other three cannot	it says wrong , never where — one build cycle per bisection step

An oracle that is not on the build path is a test, not a gate.

Five known gaps, ranked — and the first one is half a model

gap	what it blocks	what closing it takes
1. the write / store side	the epilogue; the single-LDS-buffer configuration (the reuse is invisible to the hazard analysis); the ring-1 copy carries no WAR at all	a movement whose destination is global space, a register-lifetime-vs-store hazard kind, an output operand with its own presence, a phase after the last drain — plus reverse-pair counters and a reflected peel, which today refuses rather than computes. Comparable in size to the read side.
2. tail loop and edges	K not a multiple of the chunk, free extents not a multiple of the tile: the old scaffold still serves them, so their reads and pointer state are outside the model	a per-level peel for a <i>runtime</i> -extent chunk. The hard half: drain and tail overlap under a reduction-axis storage split, and do not under a free-axis one
3. register partitioning	nothing — and that is the problem: more than one group per operand is a silent no-op at emission	the per- (operand, group) widths are already computed and nothing reads them. Carry a per-group slot list in the emit plan, return the map instead of a scalar, teach the matrix leaf to select
4. half-wave scale selector	a fallback leaf re-derives a model-level fact: half the read acts stop issuing and the register index is compacted	make the merge map <i>derivable</i> rather than supplied. Four attempts to infer the pairing inside the model were each an address assumption in disguise
5. persistent / grid-stride level	an outer time level above the reduction chunk	one extra mode with extent 0 in θ — no new level kind. What blocks it is the emitter refusing a coarser peel, and the frame arithmetic across an outer trip

The tail's block phase exists, and the verifier already *accepts* a tail as a second loop region. Nothing produces one.

Expressible is not emitted — and each of these five is recorded in the source at the line that refuses.

Two packages, three optimisations worth doing — and one bet

Tensile/LoopModel/

the schedule model: what happens *when*. θ , the peel, the ledger, the certificate. Every gate is either totality or honesty

Tensile/Lowering/

the dataflow lowering: *where* each value lives. GIR, the analyses, the passes, the emitter. Each package carries its own gap list and cost map in-tree

Compile time, measured not estimated: **8 ms** of lowering on the common shape, **165 ms** on the worst measured one (4 waves, DepthU 512, a 1936-instruction body). Kernels build one per worker, so per-kernel wall time is what a sweep pays.

ranked by win / risk	win	risk
fence slots as intervals + a difference array, not integer sets	~45 ms	low
let the verifier accept the caller's analysis manager	~14 ms	very low
hand the precomputed plan to the semantic gate	~12 ms	none

Everything below those is single-digit milliseconds. The CFG walks, the fixpoints and the token machinery are *deliberately* not on the list — they are free and they are the clearest code in the tree.

The bet: a main loop's schedule is **data** — one θ point — and every physical consequence of it (which buffer, which pointer flip, which wait, which barrier) is **derived by dataflow and certified**, never hand-placed per configuration.